

Práctica 4

Lectura sucia en dos conexiones · AdventureWorks2022

1. Problema representado en la imagen

La conexión A incrementa el precio de un producto dentro de una transacción sin confirmar. La conexión B usa **read uncommitted** para leer el precio provisional y lo guarda. A ejecuta **rollback**, pero B registra un pedido con el precio que ya no existe como valor confirmado. Esta anomalía se conoce como **lectura sucia**.

Microsoft SQL Server 2000 Programming by Example

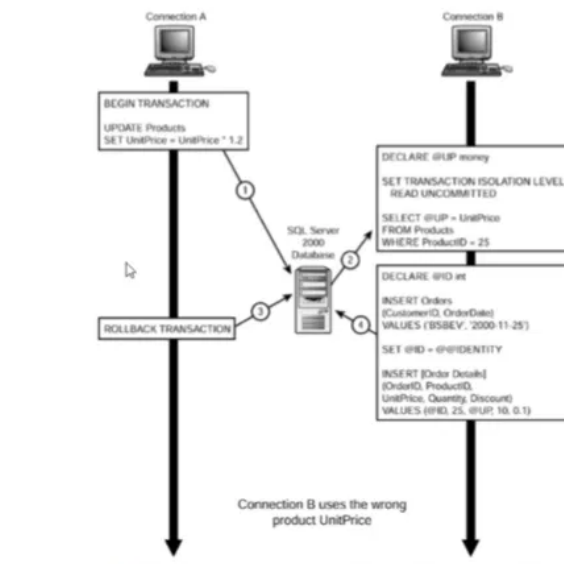


Figura original proporcionada: A modifica y revierte; B lee y usa el precio no confirmado.

Alcance: se estudia únicamente el problema de la imagen, no la actualización perdida. **Estado de resultados:** los valores de este informe son esperados, no observaciones certificadas de una instancia de SQL Server.

2. Preparación y orden de ejecución

Abra **dos ventanas de SSMS** en el mismo servidor. Abra **Practica_4_ConexionA.sql** en A y **Practica_4_ConexionB.sql** en B. Seleccione con el cursor únicamente el bloque que corresponde en cada paso, incluso su **go** final. No ejecute los archivos completos.

Orden	Ventana	Operación y resultado esperado
a00	A	Preparar tablas; precio inicial = 100.0000.
a01	A	Actualizar sin commit; precio provisional = 120.0000.
b02	B	Leer con read uncommitted; #p4_lectura = 120.0000.
a03	A	Rollback; precio confirmado = 100.0000.
b04	B	Insertar detalle 1 con precio 120.0000.
a05	A	Crear otra modificación no confirmada: 120.0000.
b06	B	Iniciar insert con read committed; puede esperar.
a07	A	Rollback; B termina con precio 100.0000.
b08	B	Consultar producto y detalles ya registrados.
b09a/b09b	B	Llenar y consultar dbo.p4_resultados.

3. Código de la conexión A

a00 · crear las tablas de prueba

Se crea una fila de producto con precio controlado de 100.0000 y dos tablas para detalles y resultados.

```
use adventureworks2022;
go
if object_id('dbo.p4_resultados', 'u') is not null
    drop table dbo.p4_resultados;
if object_id('dbo.p4_detallepedido', 'u') is not null
    drop table dbo.p4_detallepedido;
if object_id('dbo.p4_producto', 'u') is not null
    drop table dbo.p4_producto;
go
create table dbo.p4_producto (
    productoid int not null primary key,
    productoawid int not null,
    nombre nvarchar(50) not null,
    preciolista decimal(19,4) not null
);
```

Continuación del mismo bloque a00:

```
create table dbo.p4_detallepedido (
    detalleid int identity(1,1) primary key,
    productoid int not null
        references dbo.p4_producto(productoid),
    preciounitario decimal(19,4) not null,
    cantidad int not null check (cantidad > 0),
    descuento decimal(4,3) not null
        check (descuento between 0 and 1)
);
```

3. Código de la conexión A (continuación)

```
create table dbo.p4_resultados (
    escenario nvarchar(30) not null primary key,
    detalleid int not null,
    precioconfirmado decimal(19,4) not null,
    precioregistrado decimal(19,4) not null,
    diferenciaunidad decimal(19,4) not null,
    cantidad int not null,
    descuento decimal(4,3) not null,
    totalregistrado decimal(19,4) not null,
    totalreferencia decimal(19,4) not null,
    diferenciatal decimal(19,4) not null
);
go
insert dbo.p4_producto
    (productoid, productoawid, nombre, preciolista)
select top (1) 1, productid, name, 100.0000
from production.product
where listprice > 0
order by productid;
select productoid, productoawid, nombre, preciolista
from dbo.p4_producto;
go
```

a01 · modificación provisional

```
begin transaction;
update dbo.p4_producto
set preciolista = round(preciolista * 1.20, 4)
where productoid = 1;
select productoid, preciolista as precioprovisional
from dbo.p4_producto where productoid = 1;
-- dejar la transaccion abierta: no usar commit.
go
```

a03 · revertir la primera modificación

```
rollback transaction;
select productoid, preciolista as precioconfirmado
from dbo.p4_producto where productoid = 1;
go
```

3. Código de la conexión A (final)

a05 · repetir el cambio sin confirmar

```
begin transaction;
update dbo.p4_producto
set preciolista = round(preciolista * 1.20, 4)
where productoid = 1;
-- dejar la transaccion abierta durante el paso 6.
go
```

a07 · liberar la lectura de B

```
rollback transaction;
go
```

Si en b06 el comando insert permanece esperando, a07 libera el bloqueo de la fila. B puede terminar con el precio confirmado de 100.0000. Con **read_committed_snapshot** activo, B puede no necesitar esperar.

4. Código de la conexión B

b02 · leer el precio provisional

```
use adventureworks2022;
go
set transaction isolation level read uncommitted;
if object_id('tempdb..#p4_lectura') is not null
    drop table #p4_lectura;
select productoid, preciolista as precioleido
into #p4_lectura
from dbo.p4_producto
where productoid = 1;
select productoid, precioleido from #p4_lectura;
go
```

4. Código de la conexión B (continuación)

b04 · insertar el pedido incorrecto

```
set transaction isolation level read committed;
insert dbo.p4_detallepedido
    (productoid, preciounitario, cantidad, descuento)
select productoid, precioleido, 10, 0.100
from #p4_lectura;
select d.detalleid, p.preciolista as precioconfirmado,
       d.preciounitario as precioregistrado,
       d.preciounitario - p.preciolista as diferencia
from dbo.p4_detallepedido as d
join dbo.p4_producto as p on p.productoid = d.productoid
order by d.detalleid;
go
```

b06 · insertar el pedido protegido

```
set transaction isolation level read committed;
insert dbo.p4_detallepedido
    (productoid, preciounitario, cantidad, descuento)
select productoid, preciolista, 10, 0.100
from dbo.p4_producto
where productoid = 1;
-- con bloqueo tradicional esta consulta esperara al paso 7.
go
```

b08 · consultar las tablas de origen

```
-- ejecutar cuando el paso 6 haya terminado.
select productoid, preciolista as preciofinal
from dbo.p4_producto;
select detalleid, productoid, preciounitario, cantidad,
       descuento,
       cast(preciounitario * cantidad * (1 - descuento)
           as decimal(19,4)) as totalregistrado
from dbo.p4_detallepedido
order by detalleid;
go
```

4. Código de la conexión B (resultados)

b09a · materializar los resultados obtenidos

Esta instrucción calcula las diferencias por unidad y por total, y guarda ambas filas en una **tabla permanente**. Ejecutarla solo después de que b06 termine.

```
-- llenar la tabla con los resultados reales de la ejecucion.
delete from dbo.p4_resultados;
insert dbo.p4_resultados
    (escenario, detalleid, precioconfirmado,
     precioregistrado, diferenciaunidad, cantidad,
     descuento, totalregistrado, totalreferencia,
     diferenciatal)
select
    case when d.detalleid = 1
        then 'lectura sucia' else 'read committed' end,
    d.detalleid,
    p.preciolista,
    d.preciounitario,
    d.preciounitario - p.preciolista,
    d.cantidad,
    d.descuento,
    cast(d.preciounitario * d.cantidad *
        (1 - d.descuento) as decimal(19,4)),
    cast(p.preciolista * d.cantidad *
        (1 - d.descuento) as decimal(19,4)),
    cast((d.preciounitario - p.preciolista) *
        d.cantidad * (1 - d.descuento) as decimal(19,4))
from dbo.p4_detallepedido as d
join dbo.p4_producto as p on p.productoid = d.productoid
where d.detalleid in (1, 2);
go
```

b09b · leer dbo.p4_resultados

```
-- tabla permanente de resultados, util para el informe.
select escenario, detalleid, precioconfirmado,
    precioregistrado, diferenciaunidad,
    cantidad, descuento, totalregistrado,
    totalreferencia, diferenciatal
from dbo.p4_resultados
order by detalleid;
go
```

5. Tablas con resultados esperados

Todas las cifras monetarias siguientes son **resultados previstos** para el precio de laboratorio de 100.0000. Al ejecutar los archivos, las consultas b08 y b09b permiten contrastar los valores que realmente devuelve SQL Server.

Tabla 1. Estado del precio por etapa

Paso	Fuente	Precio
a00	dbo.p4_producto, inicial	100.0000
a01	A: valor sin confirmar	120.0000
b02	B: #p4_lectura.precioleido	120.0000
a03	dbo.p4_producto, confirmado	100.0000
a05	A: segundo valor provisional	120.0000
a07	dbo.p4_producto, final	100.0000

Tabla 2. Resultado de dbo.p4_detallepedido

detalleid	preciounitario	cantidad	descuento	totalregistrado
1	120.0000	10	0.100	1080.0000
2	100.0000	10	0.100	900.0000

Tabla 3. Resultado de dbo.p4_resultados

escenario	detalleid	precio confirmado	precio registrado	dif. por unidad
lectura sucia	1	100.0000	120.0000	20.0000
read committed	2	100.0000	100.0000	0.0000
escenario		total registrado	total referencia	dif. total
lectura sucia		1080.0000	900.0000	180.0000
read committed		900.0000	900.0000	0.0000

Cálculo: total registrado = precio del pedido × cantidad × (1 – descuento). El exceso de la primera operación es 20.0000 por unidad antes de aplicar el descuento y 180.0000 en total, con 10 unidades y 10 % de descuento.

6. Análisis y conclusiones

Escenario 1 · read uncommitted

La lectura realizada en b02 puede consumir 120.0000, aunque A no ha hecho commit. Después de a03, el precio confirmado es 100.0000, pero b04 guarda 120.0000 en el detalle 1. El rollback de A **no revierte el detalle confirmado por B**. La tabla de resultados muestra una diferencia total esperada de 180.0000.

Escenario 2 · read committed

En b06, B obtiene el precio al insertar en la misma instrucción, bajo **read committed**. Bajo el comportamiento tradicional de bloqueo puede esperar al rollback de a07; con versionado read_committed_snapshot puede leer la versión ya confirmada. En ambos casos, el precio esperado del detalle 2 es 100.0000 y la diferencia total es 0.0000.

Conclusión

La anomalía aparece porque una transacción usa y conserva datos de otra que todavía podrían revertirse. **read committed** impide que esa lectura use el precio no confirmado; leer el precio directamente en el insert evita reutilizar un valor provisional conservado previamente. La prueba solo demuestra este problema concreto, no todos los fenómenos de concurrencia.

Fuente de la problemática

Guerrero, F. G. y Rojas, C. E., Microsoft SQL Server 2000 Programming by Example, capítulo 13, figura 13.2 (p. 523 del libro). La adaptación usa AdventureWorks2022 y tablas de laboratorio; no reproduce literalmente el código de Northwind.